Pontificia Universidad Javeriana

Facultad de Ingeniería

Sistemas Operativos

Taller de Evaluación de Rendimiento

Martín Sanmiguel Delgado

Juan Manuel Baez Parra

Mateo Traslaviña Moreno

27 de abril. 2026

Bogotá - Colombia.

1. Introducción

El presente taller de evaluación de rendimiento tiene como objetivo comparar el comportamiento de distintos sistemas de cómputo, operando bajo distribuciones de Linux, al ejecutar diferentes algoritmos de multiplicación de matrices. El propósito central es contrastar el rendimiento de estas operaciones cuando se ejecutan de forma secuencial frente a implementaciones concurrentes (en paralelo), para posteriormente realizar un análisis a profundidad que correlacione el tiempo de ejecución con las especificaciones de hardware de cada máquina. Para garantizar la validez estadística de este análisis, se utilizará como métrica base el tiempo de ejecución promedio obtenido tras 30 iteraciones en cada escenario, a partir del cual se evaluarán indicadores clave como el *Speedup* (aceleración) y la eficiencia en la utilización de los recursos del sistema.

2. Descripción de los sistemas de computo

Para la realización del experimento se utilizaron 6 sistemas de cómputo Linux, con cuatro arquitecturas de hardware diferentes, ya que hay dos pares de máquinas que son iguales. A continuación, se muestran los componentes de hardware de cada uno de los equipos utilizados:

ID	CPU	Nuclos logicos	Frec. Máx. (GHz)	L1d/L1i (KiB)	L2 (MiB)	L3 (MiB)	RAM (GiB)	Tipo
PC-1	Intel Core i7-6500U	4	3.1	64/64	0.5	4	7.8	Portátil
PC-2	Intel Core i7-6500U	4	3.1	64/64	0.5	4	7.8	Portátil
Escritorio- 1	Intel Core i7-6700T	8	3.6	128/128	1.0	8	7.8	Desktop
Escritorio- 2	Intel Core	8	3.6	128/128	1.0	8	7.8	Desktop

	i7-6700T							
PC-Martin	Intel Core i7-13650HX	20	4.9	544/704	11.5	24	14.8	Portátil
VM	Intel Xeon Gold 6240R (VMware)	4 (vCPU)	2.4	128/128	4.0	35.8	12	VM

Además, todas las maquinas que utilizamos tenían sistema operativo Linux Ubuntu

3. Diseño experimental

Para la realización del experimento se utilizaron dos algoritmos de multiplicacion de matrices los cuales fueron los siguientes:

- **Filas por columnas (FxC):** Es el método tradicional de multiplicar matrices. el cual consiste en multiplicar cada elemento de la fila de la primera matriz por cada elemento de la columna de la segunda matriz. Sin embargo, computacionalmente es ineficiente debido a la forma en que el lenguaje C guarda los datos en la memoria un principio conocido como Row-Major Order o almacenamiento por filas [1] [2]. Al recorrer la segunda matriz por columnas, el procesador se ve obligado a “saltar” constantemente entre diferentes direcciones de memoria [1], Esto se puede ver más fácilmente en la figura 1 que nos muestra cómo es que realmente los datos de los arreglos multidimensionales son almacenados. Esto hace que la memoria caché no se aproveche [3], ya que se viola el principio de localidad espacial, obligando al sistema a buscar los datos en la memoria principal, lo cual hace que el algoritmo sea mucho más lento, asi que podríamos esperar tiempos mucho mayores de este algoritmo.

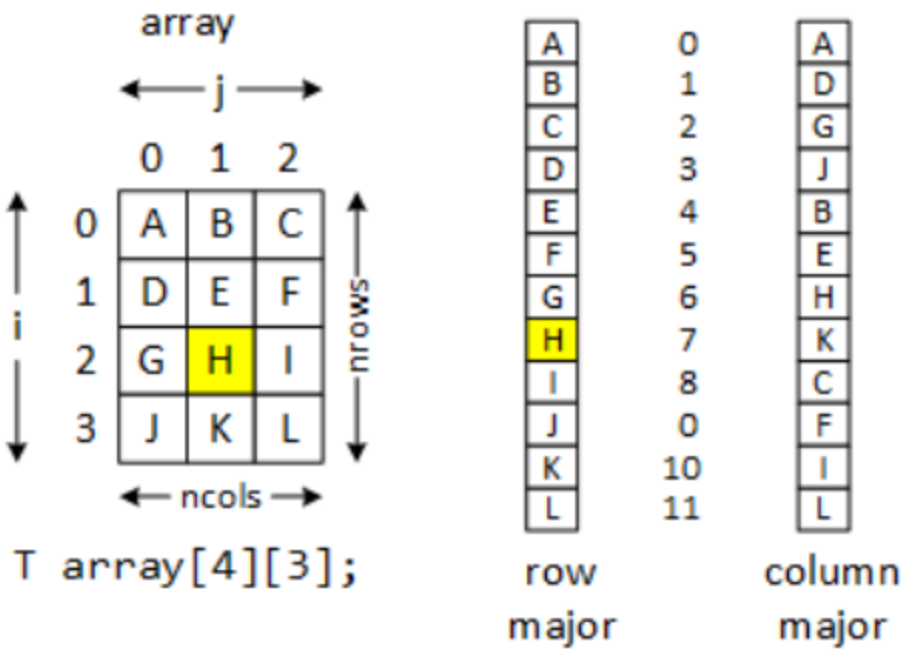

Figura 1: imagen tomada de [3] que representa Row-Major Order o almacenamiento por filas demostrando como el lenguaje C, almacena los datos de los arreglos multidimensionales

- **Filas por transpuestas (FxT):** Es una optimización pensada en cómo funciona la memoria del computador. Al transponer primero la segunda matriz, sus columnas se convierten en filas, haciendo que no se tenga que "saltar" constantemente entre direcciones de memoria aprovechando mucho mejor el principio de localidad espacial como se puede observar en la figura 1. De esta manera, cuando el procesador hace la multiplicación, lee ambas matrices de corrido, en línea recta. Todo esto genera que la memoria caché trabaja a su máxima velocidad, asi que podemos esperar un tiempo de ejecucion mucho menor comparandolo con el algoritmo Filas por columnas.

Antes de detallar las variables experimentales, es imperativo establecer la distinción a nivel de sistema operativo entre los dos mecanismos de concurrencia evaluados:

Hilo: Es una secuencia de instrucciones que el procesador ejecuta. Estos compranten memoria y necen de la necesidad de los programas de hacer varias cosas a la vez.

Proceso: Es un programa completo en ejecución. A diferencia de los hilos, los procesos son totalmente independientes y no comparten memoria.

Adicionalmente, para evaluar el comportamiento de estos algoritmos bajo distintas cargas de trabajo y niveles de concurrencia, el diseño del experimento contempla las siguientes variables:

- **Tamaño de las matrices:** Se realizarán pruebas con matrices cuadradas de dimensiones 512, 1024 y 2048.
- **Nivel de concurrencia:** Se evaluará una ejecución puramente secuencial 1 hilo y ejecuciones paralelas distribuyendo la carga de trabajo en 4, 8 y 16 hilos o procesos.
- **Mecanismos de paralelismo:** La concurrencia se abordará mediante dos implementaciones a nivel de sistema operativo: instanciación de **Hilos (POSIX)**, los cuales comparten el mismo espacio de direccionamiento de memoria, y creación de **Procesos (Fork)**, los cuales generan espacios de memoria independientes para cada bifurcación.

4. Recolección y Procesamiento de la Información

Dado que los sistemas operativos de propósito general ejecutan múltiples procesos en segundo plano que pueden alterar el rendimiento de la CPU en un momento dado, por lo tanto, se realizó el experimento varias veces para mitigar este tipo de variaciones:

Para garantizar la integridad y repetibilidad del experimento, la recolección se dividió en dos fases automatizadas:

Fase 1: Ejecución (Script Perl): Se utilizó el script lanzador.pl para automatizar la ejecución de toda la batería de pruebas en las cuatro máquinas. El script se encargó de compilar los binarios y ejecutar cada combinación exacta de variables (algoritmo, tamaño de matriz e hilos) 30 veces consecutivas. Los tiempos individuales de cada iteración se exportaron automáticamente a archivos de texto plano (.dat), generando un gran volumen de datos crudos.
Fase 2: Consolidación y visualización (Script Python): Para evitar errores de manipulación manual humana, se desarrolló un script de procesamiento de datos en Python utilizando las librerías Pandas, Seaborn y Matplotlib. Este código recorrió el árbol de directorios de las 4 máquinas, leyó cada archivo .dat y calculó el promedio aritmético exacto de las 30 iteraciones. Finalmente, el programa exportó los datos consolidados a archivos CSV y genero automáticamente las gráficas comparativas que sustentan el análisis posterior.

5. Análisis de Resultados

En esta sección se evalúa el impacto de las distintas variables de hardware y de software sobre los tiempos de ejecución. El análisis se fundamenta en el contenido obtenido por los Scripts descritos en la sección anterior

5.1. Impacto de la cantidad de hilos, cantidad de trabajo y escalabilidad

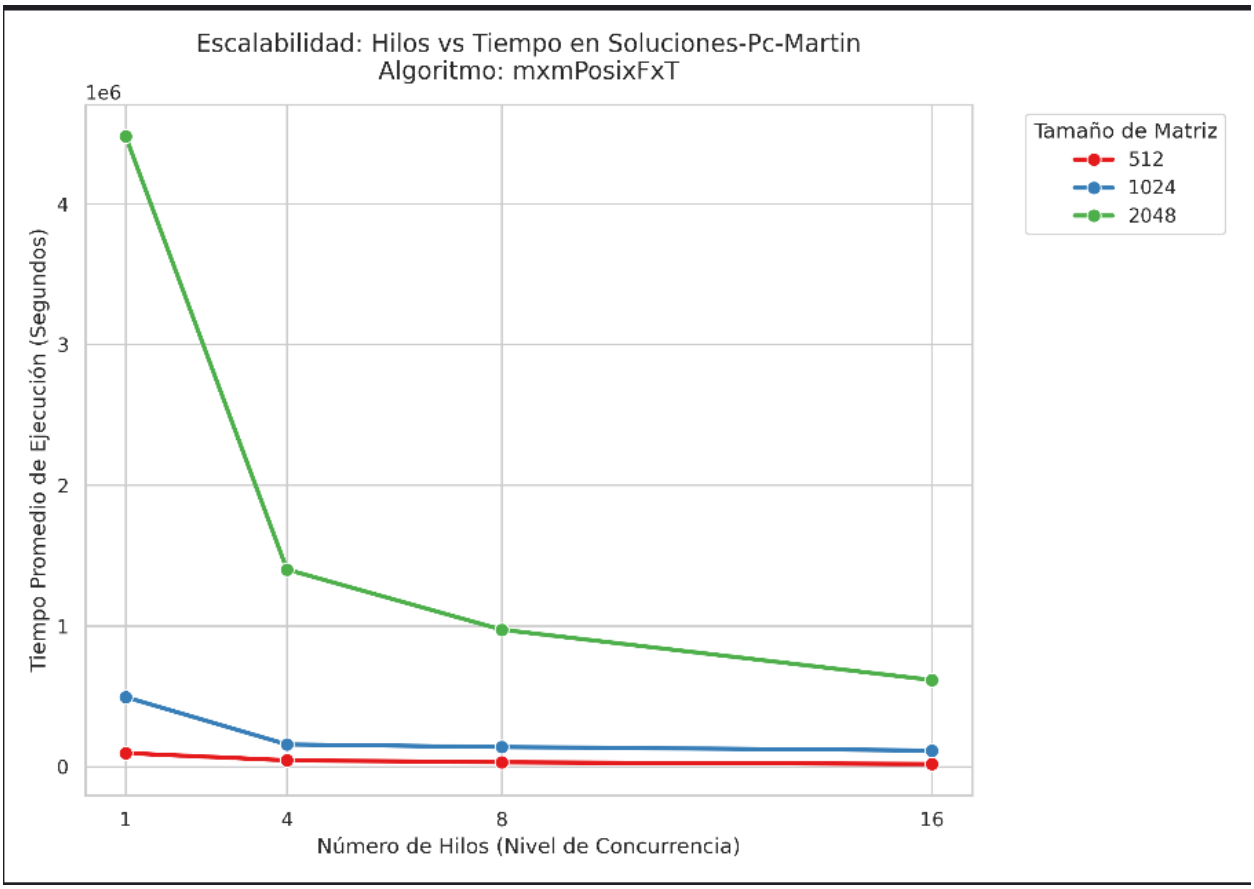

Figura 2: en esta grafica podemos observar el Overhead con claridad en el computador PC-martin, esta grafica compara los tiempos con diferentes tamaños de matrices y diferente cantidad de hilos

En la figura dos podemos observar el cambio de contexto o Overhead [4], este existe ya que para que la concurrencia sea posible, el núcleo del sistema operativo debe intervenir para crear los hilos, asignar recursos y realizar constantes cambios de contexto (osea consumir tiempo en tareas administrativas) y cuanto la tarea es muy pequeña como cuando se realiza la multiplicación de matrices de tamaño 512x512 (Figura 2) pues el sistema operativo consume demasiado tiempo creando y gestionando los 16 hilos (línea naranja), es por esto que se puede observar que es una línea casi recta, osea que aunque se tiene una eficacia muy grande con la gran cantidad de hilos la mayoría del tiempo de ejecucion es consumido por el Overhead. En este caso, el sistema operativo gasta más tiempo organizando el trabajo que los hilos ejecutándolo.

Por el contrario, en la matriz de 2048x2048, el panorama cambia drásticamente. Dado que la carga de trabajo es masiva, el costo del overhead se vuelve despreciable frente al volumen de cálculos realizados. Esto genera una escalabilidad positiva, observable en la caída pronunciada de la curva: al pasar de 1 a 4 hilos el tiempo de ejecución se reduce de forma exponencial, manteniendo una mejora significativa hasta los 16 hilos. Aquí, el sistema logra una utilización óptima del hardware, ya que el esfuerzo administrativo de la concurrencia se compensa con una reducción sustancial en el tiempo de procesamiento total.

5.2 Análisis de Saturación de Hardware (caso PC-2, Escritorio-2, VM)

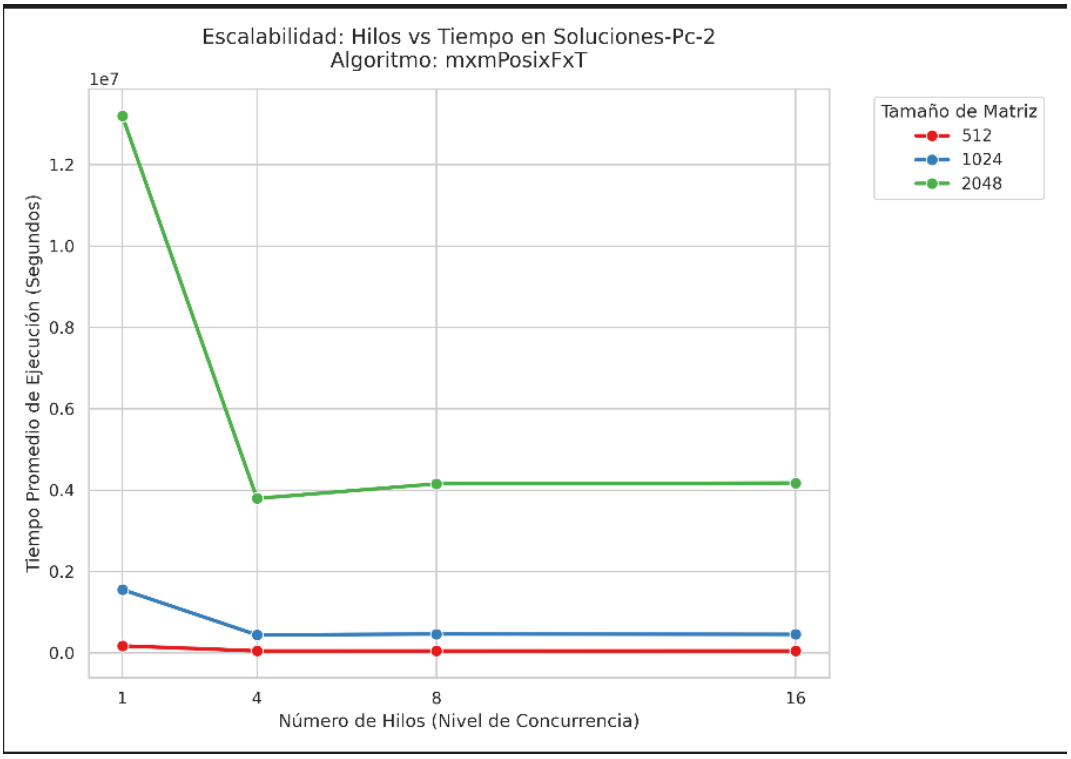

Figura 3: en esta figura evidenciar lo que pasa cuando al sistema de cómputo se le piden usar más hilos de los que tiene, esta es la gráfica de escalabilidad del Pc 2

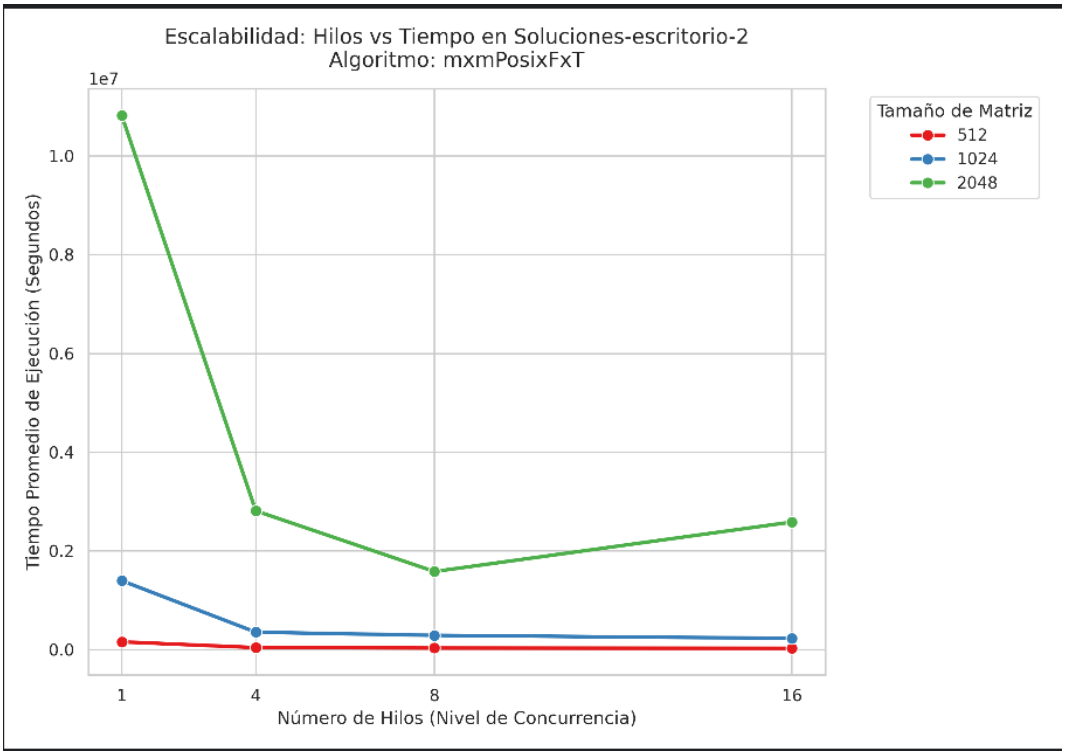

Figura 4: en esta figura evidenciar lo que pasa cuando al sistema de cómputo se le piden usar más hilos de los que tiene, esta es la gráfica de escalabilidad del escritorio 2

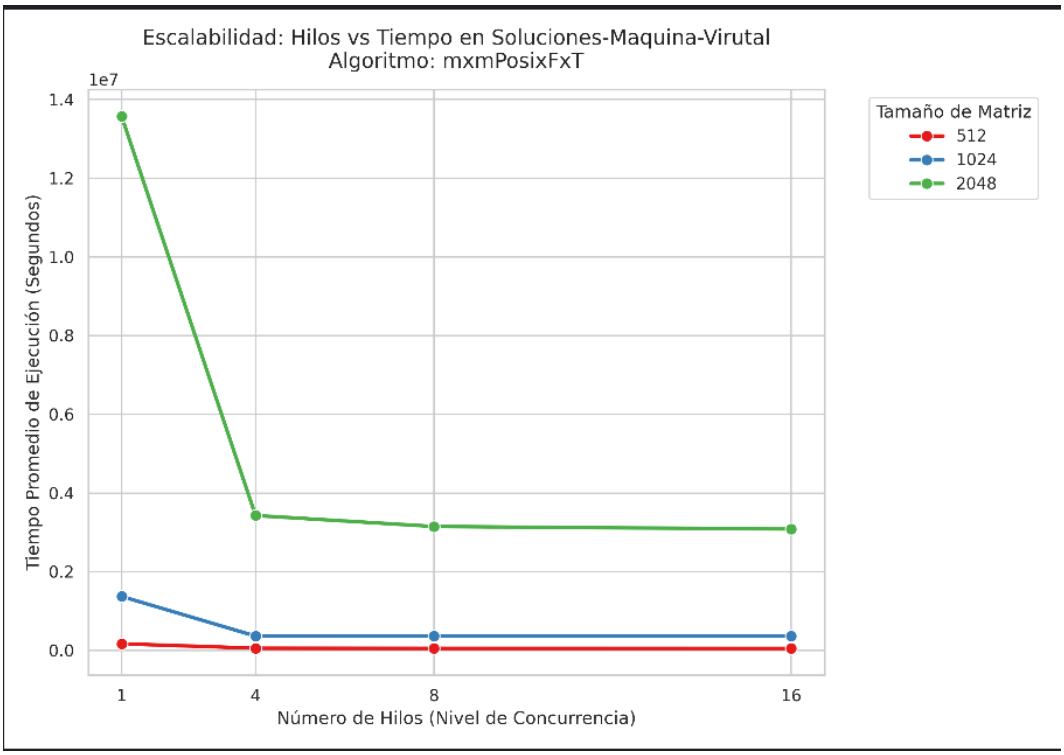

Figura 5: en esta figura evidenciar lo que pasa cuando al sistema de cómputo se le piden usar más hilos de los que tiene, esta es la gráfica de escalabilidad de la máquina virtual

Unos casos muy interesantes son los que se presentan cuando se observan las gráficas de escalabilidad del PC-2, ESCRITORIO-2 Y MAQUINA-VIRTUAL (figura 3-5). Primero podemos analizar el caso del PC-2, ya que el caso de este sistema de cómputo es similar al resto. Este equipo a diferencia otros equipos con mayor capacidad (como el Pc-Martin), este sistema cuenta con una limitación física de 4 hilos de ejecución.

En la Figura 3, se puede observar que el tiempo de ejecución disminuye correctamente al pasar de 1 a 4 hilos. Sin embargo, al incrementar la carga a 8 y 16 hilos, la curva se vuelve completamente plana. Este comportamiento se analiza bajo los siguientes puntos:

Límite de Paralelismo: Una vez que el software solicita más hilos (8 o 16) de los que el hardware puede procesar simultáneamente (4), el procesador alcanza su punto máximo de rendimiento, ya que no hay unidades de ejecución libres para procesar las nuevas tareas en paralelo, así que se llega a un límite físico del hardware.

Gestión por Tiempo Compartido: Para manejar los 16 hilos en solo 4 núcleos lógicos, el Sistema Operativo debe recurrir a la alternando trozos de tiempo del procesador entre los 16 hilos. Aunque el programa intenta ejecutar 16 tareas a la vez, el hardware solo está procesando 4 en cualquier instante dado (o se muestra con un rendimiento similar a 4).

Estabilización del Rendimiento: El hecho de que la línea sea plana indica que el sistema es y que esta no "baje" confirma la Ley de Amdahl [5][6]: el límite de la aceleración está dictado por la parte del hardware que no puede ser paralelizada más allá de sus recursos físicos o en palabras más simples el hardware que no puede ofrecer más rendimiento esta generando un cuello de botella.

Estas tres cosas son las que ocurren en el resto de los sistemas generando que la mejora de eficiencia se detenga cuando se llega al límite físico del sistema de computo.

5.3 Comparacion POSIX y FORK

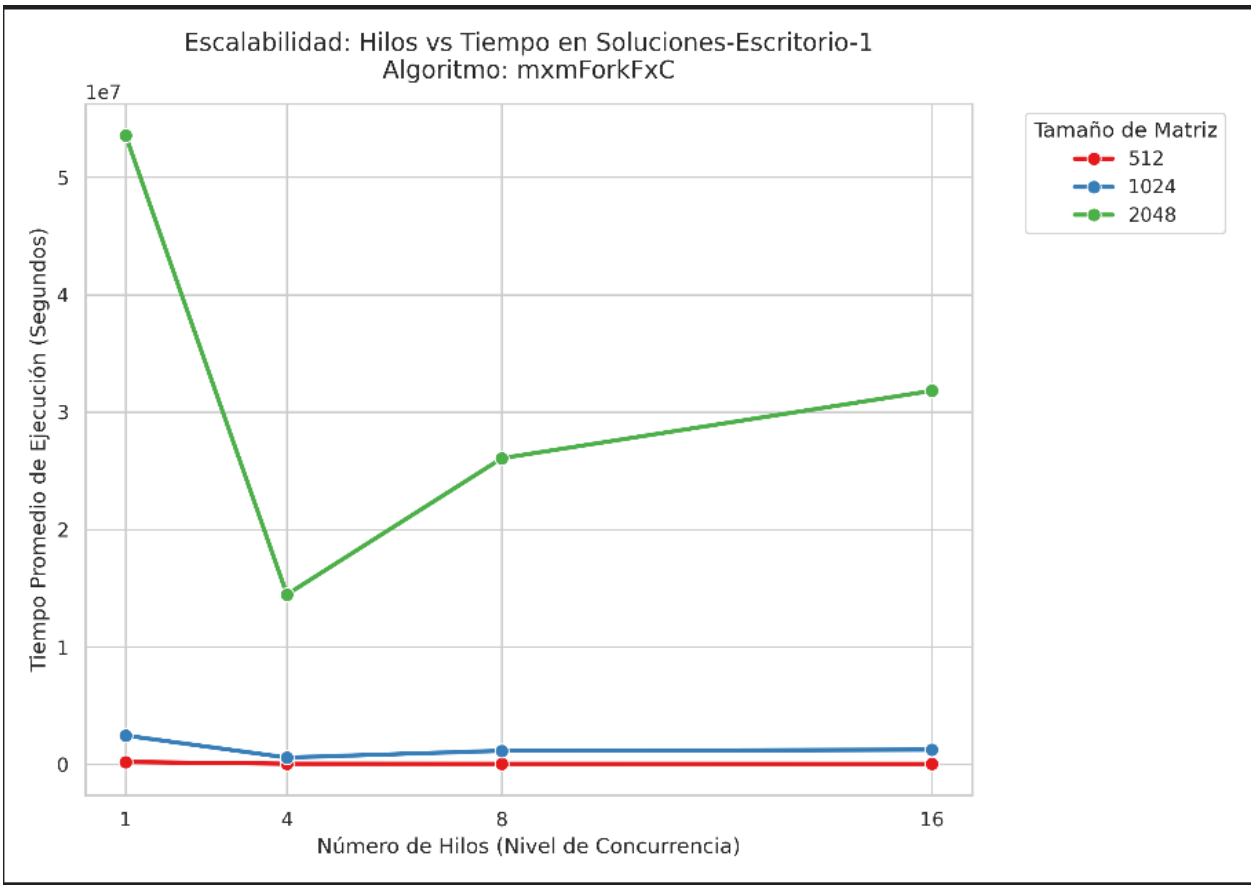

Figura 6: grafica de escalabilidad de la maquina escritorio 1 FxC con fork

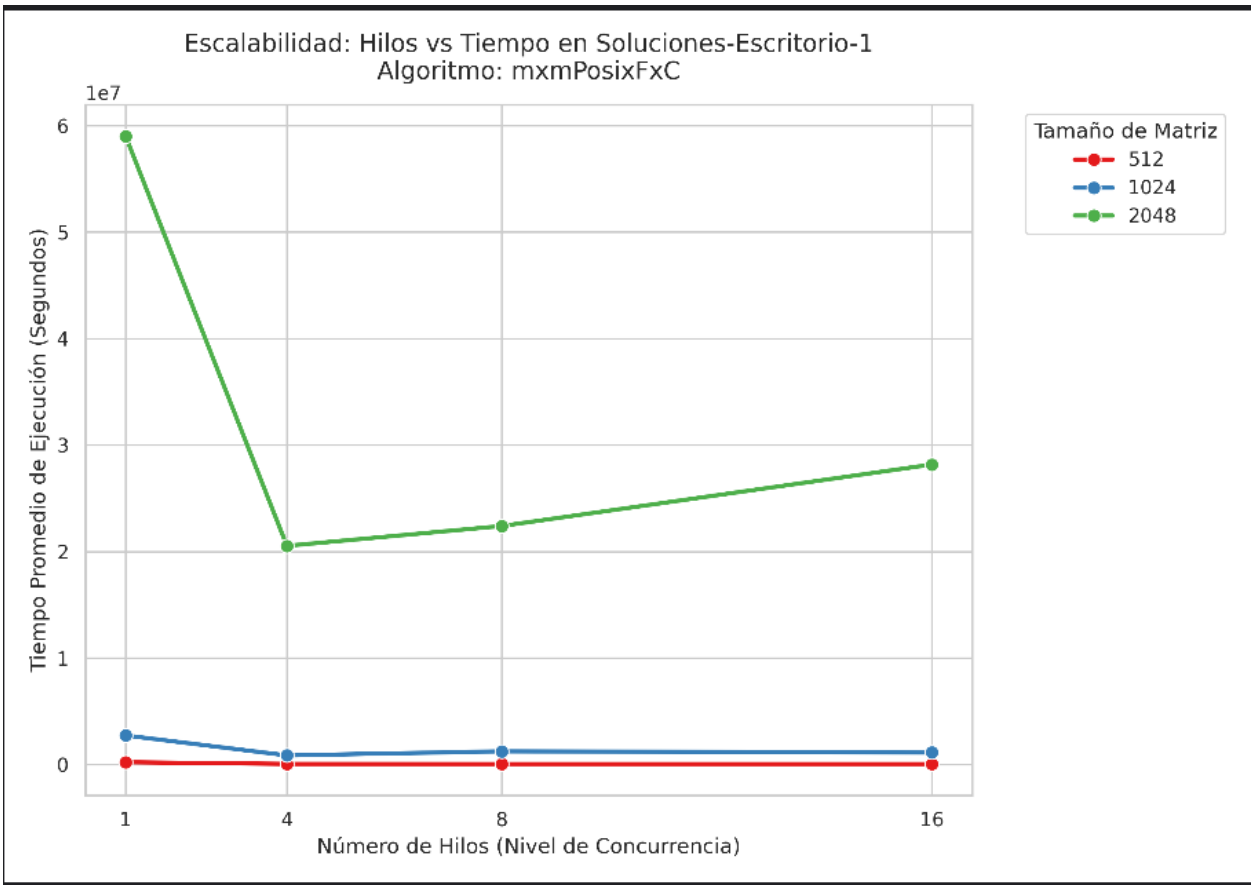

Figura 7: grafica de escalabilidad de la maquina escritorio 1 FxC con POSIX

Al contrastar los resultados obtenidos entre las implementaciones de POSIX Threads y la llamada al sistema Fork (visualizados en las Figuras 6 y 7), se evidencia una diferencia sustancial en el rendimiento, especialmente bajo cargas de trabajo elevadas (2048x2048).

Como se puede ver POSIX es más rápido que FORK y esto pasa por La superioridad técnica de POSIX se genera ya que los hilos operan dentro del mismo espacio de memoria del proceso padre. Esto permite que la comunicación y el acceso a las matrices sean directos y compartidos. Por el contrario, cada vez que se ejecuta un Fork, el sistema operativo debe crear un nuevo proceso con su propio espacio de direccionamiento independiente.

Conclusiones

1. El algoritmo FxT es consistentemente superior a FxC en todos los sistemas evaluados. Esto se debe al aprovechamiento del principio de localidad espacial en la memoria caché, evitando los saltos de memoria que caracterizan al algoritmo FxC bajo el modelo Row-Major Order de C.
2. POSIX Threads supera a Fork en todos los escenarios, especialmente en matrices grandes (2048×2048). La razón es estructural: los hilos POSIX comparten el espacio de memoria del proceso padre.

3. El overhead de concurrencia es más notable en matrices pequeñas. Para matrices de 512×512, el tiempo que el sistema operativo invierte en crear, gestionar y sincronizar hilos supera al beneficio del paralelismo, haciendo que usar 16 hilos sea incluso menos eficiente que ejecutar el algoritmo secuencialmente.
4. El gran tamaño de la carga de trabajo es para paralelismo sea beneficioso. En matrices de 2048×2048 el overhead se vuelve despreciable frente al volumen de cálculos, permitiendo observar una redu89cción exponencial del tiempo al escalar de 1 a 4 hilos, con mejoras sostenidas hasta 16 hilos en máquinas con suficientes núcleos.
5. Existe un límite físico del hardware que la Ley de Amdahl describe con precisión. En los equipos con 4 núcleos lógicos (PC-1, PC-2, VM), al solicitar 8 o 16 hilos el rendimiento se estabiliza completamente, ya que el sistema operativo debe gestionar los hilos adicionales por tiempo compartido, sin obtener ganancia real de procesamiento paralelo.
6. Las diferencias de hardware se reflejan directamente en los resultados. El PC-Martin (i7-13650HX, 20 núcleos, 4.9 GHz) demostró el mayor beneficio al escalar hilos, mientras que los equipos con arquitecturas más antiguas y menos núcleos alcanzaron su techo de rendimiento antes.

Recomendaciones

1. Usar siempre FxT sobre FxC para multiplicación de matrices en C, independientemente del sistema. La ganancia en rendimiento por el uso eficiente de caché justifica ampliamente el paso adicional de transponer la matriz.
2. Preferir POSIX Threads sobre Fork cuando se requiera concurrencia en operaciones con datos compartidos como lo es la multiplicación de matrices. Fork solo sería recomendable en escenarios donde el aislamiento total de memoria entre procesos sea un requisito de diseño.
3. Calibrar el número de hilos al número de núcleos lógicos disponibles. Superar ese límite no genera ganancia de rendimiento y añade overhead innecesario de gestión por parte del sistema operativo. Una buena práctica es consultar el número de núcleos con nproc antes de definir la concurrencia.
4. Evitar paralelismo en matrices pequeñas. Para dimensiones de 512×512 o menores, la ejecución secuencial es más eficiente. Se recomienda implementar un umbral de tamaño mínimo a partir del cual se active la concurrencia.
5. Considerar matrices de mayor dimensión (4096×4096) en máquinas con más RAM y núcleos como el PC-Martin, para observar si la escalabilidad con 16 hilos sigue mejorando o si aparecen nuevos cuellos de botella relacionados con la jerarquía de memoria.

REFERENCIAS

[1] Weber State University, "Row-Major Order," *CS 1410 Textbook*. [En línea]. Disponible en: https://icarus.cs.weber.edu/~dab/cs1410/textbook/7.Arrays/row_major.html. [Accedido: 27-abr-2026].

[2] Wikipedia, "Row- and column-major order," *Wikipedia, The Free Encyclopedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Row-_and_column-major_order. [Accedido: 27-abr-2026].

[3] W. Stallings, *Organización y arquitectura de computadores*, 8va ed. Pearson Educación, 2012, cap. 4. [Accedido: 27-abr-2026].

[4] W. Stallings, Sistemas operativos: Aspectos internos y principios de diseño, 7ma ed. Pearson Educación, 2012, cap. 3 - 4.

[5] M. E. Rodríguez García, "Principios cuantitativos del diseño de computadoras," Universidad Tecnológica de la Mixteca, 2009. [En línea]. Disponible en: https://www.utm.mx/~merg/AC/2009/2.7-principios_cuantitativos.html. [Accedido: 28-abr-2026].

[6] EcuRed, "Ley de Amdahl," EcuRed. [En línea]. Disponible en: https://www.ecured.cu/Ley_de_Amdahl. [Accedido: 28-abr-2026].